

CO 3

```
import java.util.*;

class Edge implements Comparable<Edge> {

    int src, dest, weight;

    Edge(int s, int d, int w) {
        src = s;
        dest = d;
        weight = w;
    }

    public int compareTo(Edge e) {
        return this.weight - e.weight;
    }
}

public class CDNOptimizer {

    static final int V = 5;

    static String[] server = {"S1", "S2", "S3", "S4", "S5"};

    // BFS Traversal
    static void BFS(int[][] graph, int start) {

        boolean[] visited = new boolean[V];
```

```
Queue<Integer> q = new LinkedList<>();

visited[start] = true;

q.add(start);

while (!q.isEmpty()) {

    int u = q.poll();

    System.out.print(server[u]);

    boolean hasNext = false;

    for (int i = 0; i < V; i++) {

        if (graph[u][i] > 0 && !visited[i]) {

            visited[i] = true;

            q.add(i);

            hasNext = true;
        }
    }

    if (!q.isEmpty())
        System.out.print(" -> ");
}

System.out.println();
```

```
}
```

```
// DFS Traversal
```

```
static void DFSUtil(int[][] graph, int v, boolean[] visited)
{
```

```
    visited[v] = true;
```

```
    System.out.print(server[v]);
```

```
    boolean next = false;
```

```
    for (int i = 0; i < V; i++) {
```

```
        if (graph[v][i] > 0 && !visited[i]) {
```

```
            System.out.print(" -> ");
```

```
            DFSUtil(graph, i, visited);
```

```
        }
```

```
    }
```

```
}
```

```
static void DFS(int[][] graph) {
```

```
    boolean[] visited = new boolean[V];
```

```
    DFSUtil(graph, 0, visited);
```

```
    System.out.println();
```

```
}
```

```

// Find Parent for Union-Find
static int find(int[] parent, int i) {

    if (parent[i] == i)
        return i;

    return find(parent, parent[i]);
}

// Union
static void union(int[] parent, int x, int y) {

    int xset = find(parent, x);

    int yset = find(parent, y);

    parent[xset] = yset;
}

// Kruskal MST
static void kruskalMST(ArrayList<Edge> edges) {

    Collections.sort(edges);

    int[] parent = new int[V];

    for (int i = 0; i < V; i++)
        parent[i] = i;

    int count = 0;

```

```
int mstCost = 0;

System.out.println("\nMinimum Spanning Tree:\n");

for (Edge e : edges) {

    int x = find(parent, e.src);

    int y = find(parent, e.dest);

    if (x != y) {

        System.out.println(server[e.src] + " - "
            + server[e.dest] + " = "
            + e.weight);

        mstCost += e.weight;

        union(parent, x, y);

        count++;

        if (count == V - 1)
            break;
    }
}

System.out.println("\nTotal MST Cost = " + mstCost);
}
```

```
public static void main(String[] args) {

    System.out.println("=== MEDIAVAULT CDN  
OPTIMIZATION ===\n");

    System.out.println("Servers Connected:");

    for (String s : server)
        System.out.print(s + " ");

    System.out.println("\n");

    int[][] graph = {

        {0, 4, 2, 0, 0},
        {4, 0, 0, 0, 0},
        {2, 0, 0, 3, 0},
        {0, 0, 3, 0, 1},
        {0, 0, 0, 1, 0}
    };

    System.out.println("BFS Traversal:\n");

    BFS(graph, 0);

    System.out.println("\nDFS Traversal:\n");

    DFS(graph);

    ArrayList<Edge> edges = new ArrayList<>();
```

```
edges.add(new Edge(3, 4, 1)); // S4-S5
edges.add(new Edge(0, 2, 2)); // S1-S3
edges.add(new Edge(2, 3, 3)); // S3-S4
edges.add(new Edge(0, 1, 4)); // S1-S2

kruskalMST(edges);

System.out.println("\nNetwork Optimized
Successfully");

System.out.println("\nPerformance Summary:\n");

System.out.println("BFS :  $O(V + E)$ ");

System.out.println("DFS :  $O(V + E)$ ");

System.out.println("Kruskal :  $O(E \log E)$ ");

System.out.println("\nProgram Executed
Successfully.");
    }
}
```